



BOOTCAMP FRONTEND & REACT · SEMANA 3

JavaScript

Clase 7 — La tienda empieza a pensar · variables, funciones y
map/filter/reduce

4 h 30 · L·M·V

Proyecto TechCart

STEEPCLY

PLAN DE HOY · VIENEN DE JAVA, ASÍ QUE LA LÓGICA VUELA

Agenda

1. Repaso + 2 difíciles + el salto
group y modo oscuro · de estilar a PENSAR

~30'

2. Variables y tipos
const / let · tipado dinámico

~25'

3. Operadores y control
=== vs == · if · ternario

~32'

4. Funciones
declaración vs arrow (= lambda)

~28'

5. Arrays y objetos
el catálogo como datos · js/main.js

~33'

6. map / filter / reduce
los Streams de Java, en JS

~50'

7. Arranque ES6+
template literals · desestructuración

~35'

8. Cierre + 5 tareas
resumen, lecturas y anticipo

~16'



Receso de 15 min entre el bloque 4 y el 5 · la "cancha" de hoy: la **consola del navegador**

Repaso exprés · Clase 6

Dos de memoria

¿Qué genera declarar `--color-marca` en `@theme`?

¿Qué significa el prefijo `dark:`?

El salto más grande del curso

De estilar... a pensar

Hasta hoy, la tienda se **ve**: profesional, responsive, con modo oscuro. Pero hagan clic en "Agregar al carrito"... y no pasa **nada**. Es una foto bonita. Hoy le damos un **cerebro**: empieza **JavaScript**.

Y buena noticia para los de Java: gran parte ya la saben — solo cambia cómo se escribe.

EN JS NO SE DECLARA EL TIPO — LO PONE EL VALOR

Guardar datos: **const** y **let**

const

no se puede reasignar · = **final de Java** · úsala por defecto

let

variable que sí puede cambiar de valor

~~**var**~~

la forma vieja · no la usen

JAVA

```
int precio = 1999;  
final double IGV = 0.18;
```



JAVASCRIPT

```
let precio = 1999;  
const IGV = 0.18;
```

typeof dice el tipo · EN JS HAY UN SOLO TIPO NÚMERO (NO INT/DOUBLE)

Los tipos

string

texto, entre comillas

number

enteros y decimales, juntos

boolean

true / false

null · undefined

"nada a propósito" · "sin valor aún"



Tipado dinámico: una variable puede cambiar de tipo, y JS no se queja.

```
let dato = 5 // number
dato = "cinco" // string... y JS lo
permite
```

Es flexible, pero fuente de bugs. La solución tiene nombre: **TypeScript** (en unas semanas).

ARITMÉTICOS Y LÓGICOS = IDÉNTICOS A JAVA · EL + ADEMÁS JUNTA TEXTOS

El gran tema: `===` vs `==`

`===``"5" === 5 → false`

Compara **valor Y tipo**. Es lo correcto. **Úsalo SIEMPRE** (y `!==` para distinto).

`==``"5" == 5 → true ⚠`

Convierte los tipos a **escondidas** para comparar.
Causa bugs. **Olvídalo.**

CASI IGUAL A JAVA · ¿BUCLES PARA RECORRER LISTAS? DESPUÉS DEL RECESO, CON ARRAYS

Control: if y el ternario

```
if (precio > 1000) {  
  console.log("Producto caro")  
} else {  
  console.log("accesible")  
}
```

y su versión corta: el ternario

```
const etiqueta = precio > 1000  
  ? "caro"  
  : "accesible"
```

"¿condición? valorSí : valorNo" — mismo ? :
que Java.

 EJERCICIO 1 · MÁX. 5 MIN · EN LA CONSOLA, TODAVÍA SIN ARRAYS

Etiquetar un producto

- 1) Con un **ternario**: etiqueta = "caro" si precio > 1000, si no "accesible".
- 2) Con un **if/else**: disponible = "Agotado" si stock === 0.
- 3) Imprime todo junto con +.

```
// solución
const precio = 1499, stock = 0
const etiqueta = precio > 1000 ? "caro" : "accesible"
let disponible
if (stock === 0) disponible = "Agotado"
else disponible = "Disponible"
console.log(precio + " → " + etiqueta + " · " +
disponible)
```

LA ARROW ES LA MONEDA DE MAP/FILTER/REDUCE Y DE TODO REACT

Funciones y arrow functions

CLÁSICA (COMO UN MÉTODO)

```
function conIGV(precio) {  
  return precio * 1.18  
}
```



ARROW (MODERNA)

```
const conIGV = (precio) => precio *  
1.18
```

 **Java:** es la **lambda**: precio -> precio * 1.18.
Una línea → sin return.

Arrays y objetos

array – lista ordenada (índice desde 0)

```
const cats = ["laptops", "audio"]  
cats[0] // "laptops"  
cats.length // 2
```

objeto – datos con nombre (= un POJO)

```
const p = { nombre: "iPhone", precio: 1099  
}  
p.nombre // "iPhone"
```

```
// el catálogo como array de objetos  
const productos = [  
  { nombre: "MacBook Pro", precio: 1999, stock: 5  
},  
  { nombre: "iPhone 13", precio: 1099, stock: 8 },  
  { nombre: "iPad Mini", precio: 499, stock: 0 },  
]
```

🔗 `JAVA: PRODUCTOS.STREAM().MAP().FILTER().REDUCE()` — EN JS, SIN EL `.STREAM()`

El corazón del día: **map** · **filter** · **reduce**

map

transforma cada elemento →
array del **MISMO** tamaño

filter

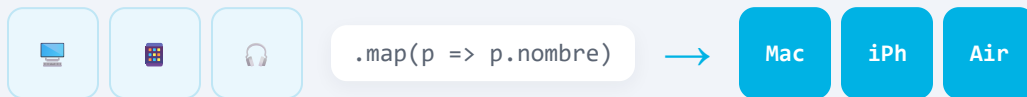
deja pasar los que cumplen →
array **MÁS CHICO**

reduce

combina todo →
UN solo valor

ENTRA UN ARRAY, SALE UN ARRAY DEL MISMO TAMAÑO, TRANSFORMADO

map — transformar



```
const nombres = productos.map(p => p.nombre)
const conIGV = productos.map(p => p.precio * 1.18)
```

LA ARROW DEVUELVE TRUE/FALSE (UNA PREGUNTA) · SALE UN ARRAY MÁS CHICO

filter — quedarse con algunos



```
const caros = productos.filter(p => p.precio > 1000)
const disponibles = productos.filter(p => p.stock > 0)
```

Se **encadenan** como los Streams: `productos.filter(...).map(...)`

UN ACUMULADOR QUE RECORRE EL ARRAY Y TERMINA EN UN NÚMERO · ¡EL TOTAL DEL CARRITO!

reduce — combinar en un valor



```
const total = productos.reduce((suma, p) => suma + p.precio, 0)  
// acumulador ↑ ↑ elemento ↑ inicio
```

⚠ Si el total sale undefined, casi seguro olvidaron el **valor inicial** (ese 0).

Template literals: adiós a los +

ES6+

lo que hacíamos – concatenar con +

```
console.log("El " + p.nombre + " cuesta $" +  
p.precio)
```



template literal

```
console.log(`El ${p.nombre} cuesta  
${p.precio}`)
```

El valor va **en su lugar**, con \${ }. Se acabó el baile de comillas y +.

Desestructuración

ES6+

lo largo

```
const nombre = p.nombre  
const precio = p.precio
```



desestructurado – una línea

```
const { nombre, precio } = p  
  
// también con arrays, por posición:  
const [primero, segundo] = productos
```

LO QUE VIMOS HOY · CUÁNTO DE ESTO YA SABÍAN DE JAVA

Resumen

const / let

tipado dinámico · const por defecto

=== (siempre)

compara valor y tipo · nunca ==

if · ternario

el ternario y for-each de Java

arrow functions

= las lambdas de Java

arrays y objetos

el catálogo como datos

map · filter · reduce

los Streams, más directos

PARA CASA · TODAS SOBRE EL ARRAY PRODUCTOS · LA QUE NADIE LOGRE LA VEMOS JUNTOS

5 tareas

FÁCIL

Solo los nombres: con map, un array con el nombre de cada producto.

INTERMEDIO

Ofertas en stock: con filter, precio < \$600 && con stock.

INTERMEDIO

El más caro: con reduce, el producto de mayor precio.

DIFÍCIL

Total de una categoría: filter + reduce — smartphones en stock, valor total.

DIFÍCIL

resumenCarrito(items): devuelve un objeto { cantidad, total } — el carrito de clases futuras.

Próxima clase

La tienda cobra vida

Hoy le dimos un cerebro, pero todo vive en la **consola** — el usuario no lo ve. La próxima clase cerramos ES6+ (spread, módulos) y damos el paso mágico: el **DOM**. El catálogo se va a **pintar solo** en la página desde el array productos, y los **botones por fin responden** al clic. La tienda deja de ser una foto y empieza a **reaccionar**.

Pista: en la consola, escriban `document.querySelector("h1")` y miren lo que devuelve. Acaban de tocar el **DOM**.